1. Declaration

I, Alan Jude Thomas, declare that this assignment, titled *A1.3: Basic App Dev V2* , is my own original work and has not been copied from any other source except where explicitly acknowledged. I have not engaged in plagiarism, collusion, or any other form of academic misconduct in the preparation and submission of this assignment. All sources of information and data used in this assignment have been properly cited and referenced in accordance with the prescribed guidelines. I have not used unauthorized assistance in the preparation of this assignment and have not allowed any other student to copy my work. I am aware that any breach of academic integrity may result in disciplinary action as per the policies of Monash University, which may include failing this assignment or the course, and further academic penalties.

Signature: _____Alan Jude Thomas_____ Date: 13/09/2026

_____

2. Github Check

Enter your Github details here.

Github Username *Enter your username here*	atho0121
Repository Shared? *Have you started and shared your assignment repository with your tutor yet?*	yes

3. Self-Evaluation

Rate your performance for each criteria. Put a ✅(tick) in the box where you think your work belongs.

Criteria	Exceeds Expectations	Meets Expectations	Needs Improvement	Fail to meet expectations
BR (C.1): Authentication		✅		
BR (C.2): Role-based authentication		✅		
BR (C.3): Rating		✅		
BR (C.4): Security		✅		

4. Screen Recording of BRs

Create a 3 minute video showing your basic web application in action! Upload this video to your Google Drive and put the link here (ensuring that you have updated the access list so its not private).

video1055760009.mp4

5. Reflections: Implementation of C.4 Security

If you have implemented BR C.4, in less than 200 words describe the approach that you have taken to implementing Security in your application. What security flaws were you trying to prevent and what security measures have you implemented to fix those flaws? How do you know that these measures will help prevent those issues from happening? Optionally you can cite external sources to provide evidence for your claim.

To address BR C.4, I implemented several client-side security practices appropriate for a front-end-only SPA. First, all user input across the Register, Login, Contact, and Rating features is validated before use — required fields, email format, password strength, and numeric range checks (ratings must be an integer 1–5) — preventing malformed or incomplete data from being processed or stored.

Second, to protect against Cross-Site Scripting (XSS), I avoided using Vue's `v-html` directive anywhere in the application. All dynamic content, including user-submitted names and messages displayed in the Admin dashboard, is rendered using Vue's default `{{ }}` text interpolation, which automatically HTML-escapes output. This means that if a malicious user entered a value like `<script>alert('xss')</script>` as their name, it would render as literal text on the page rather than executing as code — I tested this directly to confirm the behaviour.

Third, passwords are never stored in plaintext. I used the Web Crypto API to SHA-256 hash passwords before writing them to localStorage, and login failures return a single generic error message regardless of whether the email or password was incorrect, to avoid revealing which accounts exist (user enumeration).

6. Reflections: Challenges

What has been the most challenging part of this assignment for you? How has this stretched you as a programmer?

The most challenging part of this assignment was implementing authentication and role-based access without a backend to rely on. Deciding how to handle passwords responsibly on the client side was a real design problem — I chose to hash passwords using the Web Crypto API's SHA-256 function rather than storing them as plaintext, while being upfront in my code comments that this isn't a substitute for proper server-side salted hashing. Debugging an early blank-page issue also pushed me to read error messages more carefully rather than assuming the problem was more complex than it was — the actual cause was a missing default export in

my router file.

Connecting the rating feature to authentication was another stretch: making sure a rating was tied to a specific logged-in user, updated correctly rather than duplicating, and reflected live in the average shown to all users required thinking carefully about shared reactive state across components. Testing my own XSS protection by deliberately submitting a `<script>` tag as a username — and confirming Vue's escaping rendered it harmlessly — was a good reminder that security claims should be verified, not assumed.

7. Declaration: Additional Help

Any tools that you used (including Gen AI or existing code reuse) must be declared here.

Note: GenAI is not allowed for coding purposes in any assignment,

However, you may use GenAI for brainstorming and problem solving. You need to declare all such uses here. One row per help used.

Name	Description
Claude	*Used to explain Vue Router concepts (`createRouter`, `createWebHistory`, navigation guards) and help debug a blank-page rendering issue by interpreting browser console error messages and reasoning through possible causes.* *Used to review code I had already written (components, views, stores, validation logic) for bugs and improvements — e.g. identifying a race condition in an async admin-seeding function, a phone validation regex gap, inconsistent ARIA attributes, and a missing null-safety check on a status field.* *Used to think through the design and security approach for the authentication feature (BR C.1) before I wrote the code — e.g. discussing password hashing options, generic login error messages to avoid user enumeration, and why a pure client-side SPA can't replicate real backend security.* *Used to discuss how to structure role-based authorization (BR C.2) conceptually — route meta fields and navigation guards — before implementing it myself.* *Used to help verify my BR C.4 (security) implementation was sound by suggesting a manual test (submitting a `<script>` tag as input) to confirm Vue's default output escaping actually prevents XSS, rather than just assuming it.* *Used to help draft and refine Git commit messages summarizing changes I had made, following conventional commit format.* *Used to help word and structure written report sections (Reflections, security writeup) based on what I actually built and tested.*

Alan Jude Thomas
36493546
